최기환

Frontend Engineer

rlghks3004@gmail.com GitHub gihwan-dev.tistory.com

DB 성능 모니터링 제품의 프론트엔드와 공통 라이브러리를 개발합니다. 제품 규칙과 검증 절차를 AI 작업에 반영해 여러 제품의 개발과 고객 요청 처리를 병행합니다.

경력

주식회사엑셈 / 프론트엔드 개발자

2024.11 - 현재

MaxGauge Dashboard: 통합 인스턴스 모니터링

입사 초기부터 담당

`React / TypeScript / TanStack Query`

여러 DB 인스턴스의 상태와 알람을 한 화면에서 확인하는 제품입니다. 입사 초기부터 Vue Dashboard 6.0을 전담해 기획과 디자인, 개발을 맡았고 React로 이전했습니다.

- **고객사별 맞춤 개발에서 공통 기능 조합으로 전환**
  - 고객사별 기능이 플래그로 쌓여 코드를 수정할 때마다 다른 고객사 기능에 미치는 영향까지 점검해야 했음
  - 모든 고객사에 제공할 통합 모니터링 제품으로 방향을 정하고 요청받은 기능을 공통 제품에 추가
  - 고객사별 설정에서 필요한 기능을 선택해 같은 구현을 기능 조합별로 제공
- **React 디자인 시스템을 기준으로 화면 개발 통일**
  - 별도 기획자와 디자이너 없이 화면을 만들면서 작업마다 디자인과 테마가 달라졌음
  - 사내 React 디자인 시스템을 활용하도록 이전하고 공통 컴포넌트와 디자인 토큰으로 화면 구현 기준을 마련
- **API 변경에 대비해 데이터 의존성과 검증 구조 정비**
  - 화면과 강하게 결합된 전역 변수가 전면 변경 예정인 API에 의존하고 있었음
  - React 전환 후에도 구조를 정비하며 알람과 인스턴스 전역 스토어를 서비스별 조회, 변환 훅으로 대체
  - 사용자 조작과 화면 결과를 확인하는 테스트를 작성해 구조 변경 시 기존 동작을 검증

MaxGauge 차세대: DB 성능 분석

`React / TypeScript / TanStack Query / ECharts`

SQL과 세션, 데이터베이스 객체의 정보를 살펴보는 성능 분석 제품입니다. 초기에는 위젯 빌더와 SQL Detail, Object Detail의 화면 개발과 API 연동을 맡았습니다. 기획 변경 이후에는 제품 전반을 검토하고 수정하며 공통 구조와 개발 절차를 정비하고 있습니다.

- **분석 위젯 설정과 차트, 테이블 미리보기 개발**
  - 차트 미리보기 안에 데이터 준비와 표시 로직이 섞여 있어 테이블 등 다른 표현 방식을 추가하기 어려웠음
  - 데이터 준비와 화면 표시를 분리하고 시각화 종류별 렌더러를 두어 같은 설정 흐름에서 차트와 테이블 미리보기를 제공
- **22개 화면의 테이블 동작과 컬럼 정의 통일**
  - 화면별 테이블 구현이 달라 정렬과 필터 동작, 컬럼 이름과 단위가 제각각이었음
  - 범용 동작은 공통 엔진에, MaxGauge의 컬럼과 상세 화면 연결 규칙은 어댑터에 두고 화면은 컬럼과 데이터를 선언하도록 분리
  - 필요한 엔진 변경과 22개 화면 전환을 직접 수행해 테이블 동작을 통일
  - 259개 컬럼 정의를 한곳에 모아 이름과 타입 변경을 각 화면에 반복해서 반영할 필요를 없앰

MaxGauge 차세대 (계속)

- **차트에서 선택한 SQL을 앞 페이지 조회 없이 표시**
  - 차트에서 고른 SQL이 목록 뒤쪽에 있으면 앞 페이지를 모두 받은 뒤에야 해당 행을 표시했음
  - 서버가 알려주는 목표 페이지를 먼저 받고 앞뒤 데이터는 스크롤할 때 조회하도록 변경
  - 엔진에는 미조회 구간의 가상 공간과 행 이동 기능을 추가하고, 조회와 캐시는 화면이 관리하도록 책임을 구분
  - 실데이터 20,212건의 한 사례에서 요청 178건을 2건으로, 선택 행 표시까지 걸린 시간을 212.5초에서 11.8초로 줄임(당시 전후 각 1회 비교)
- **차트 선명도를 해치던 리사이즈 최적화 제거**
  - 리사이즈 중 캔버스를 늘리고 200ms 뒤 다시 그리는 처리 때문에 글자와 점이 찌그러졌음
  - CPU 4배 감속 조건에서 21만 점 Scatter의 끊김을 확인했지만, 출시 전 드문 대용량 조건을 위해 일반적인 화면의 선명도를 포기할 단계는 아니라고 판단
  - 캔버스 확대와 지연 처리를 제거하고 requestAnimationFrame으로 프레임당 한 번 실제 크기에 맞춰 다시 그리도록 변경
- **화면 규약을 AI 작업에 적용하는 스킬 작성**
  - 리뷰에서 반복 확인하던 화면 구성과 공통 컴포넌트 사용 규칙을 스킬로 정리
  - 화면을 만들거나 수정할 때 AI가 같은 규약과 참조 구현을 확인하도록 연결
  - 스킬 도입 이후 해당 규약에 대한 리뷰 지적이 더 이상 나오지 않음
- **변경 전후의 동작을 영상으로 비교하는 리뷰 도입**
  - 스크롤과 드래그처럼 움직임이 중요한 변경은 스크린샷만으로 차이를 판단하기 어려웠음
  - 같은 조작을 변경 전후에 자동 녹화하고 비교 영상으로 만드는 스킬을 작성
  - 리뷰어가 직접 재현하지 않고 영상으로 동작 차이를 확인할 수 있게 함
- **공유 테스트 서버 밖에서 검증할 수 있는 실행 환경 구성**
  - 공유 테스트 서버는 데이터가 부족하거나 다른 팀이 함께 사용해 필요한 빌드를 올리고 부하를 주기 어려웠음
  - 백엔드 서비스와 DB, 수집 데몬의 구성부터 기동과 로그인 확인까지 스크립트와 스킬로 정리
  - 원격 Linux에서 기동과 로그인, DB 접속을 확인해 공유 서버 밖에서도 제품을 검증할 수 있게 함
- **CI 실행 비용을 측정해 파이프라인 시간 단축**
  - 테스트 증가로 CI가 느려져 러너 분리를 검토했으나, 큐 대기 중앙값이 1.6초인 것을 확인하고 실행 단계의 비용을 점검
  - Storybook을 미리 빌드해 제공하고 테스트별 실행 환경을 나누는 등 불필요한 준비와 중복 실행을 줄임
  - 당시 파이프라인 시간 중앙값이 198초에서 94초로 감소(변경 전 20회, 후 4회 비교, 재시도와 캐시 재생성 제외)

exem-table: 사내 React 테이블 라이브러리 `2025.06경 인수 - 현재`

`React / TypeScript / TanStack Table / Vitest`

정렬과 검색, 행 선택 같은 동작을 여러 제품에서 재사용하는 라이브러리입니다. 차세대 프로젝트의 기존 컴포넌트를 인수해 API와 렌더링 구조, 테스트를 개선했습니다. 현재 MaxGauge 차세대와 통합 대시보드에서 사용하는 라이브러리를 맡고 있습니다.

- **기능별 API와 상태 관리 주체 정리**
  - 기능마다 API가 달랐고 외부와 테이블 내부에 상태가 중복돼 동기화가 복잡했음
  - 관련 옵션을 기능별로 모으고, 외부에서 상태를 관리하는 제어 방식과 엔진이 관리하는 비제어 방식으로 구분
  - 중간 상태 계층을 제거해 사용 제품과 테이블 엔진 사이의 상태 동기화 경로를 줄임

exem-table (계속)

- **가상화를 기본으로 하는 렌더링 구조로 전환**
  - 기존 table 기반 구조는 컬럼 리사이즈나 이동 UI를 추가할 때 DOM 배치에 제약이 있었음
  - 가상화 여부에 따라 기능을 두 번 구현하고 검증하는 부담이 커, 가상화의 제약을 감수하고 한 경로로 관리하기로 결정
  - div 기반으로 전환하고 가상화를 기본 경로로 고정해 컬럼 고정과 행 확장도 같은 구조에서 처리
- **리팩터링 이후에도 사용자 동작을 검증하는 테스트로 전환**
  - 초기 테스트가 내부 구현에 의존해 렌더링 구조를 바꾸자 함께 깨지는 문제를 겪음
  - 공개 API로 테이블을 렌더링하고 사용자 조작에 따른 DOM 결과를 확인하도록 통합 테스트를 재작성
  - 실제 브라우저에서 조작 결과를 검증하는 테스트를 쌓아 이후 엔진 전환에서도 기존 동작을 확인
- **TanStack Table v9 전환과 필요한 상태만 구독하는 구조 적용**
  - 테이블 최상위 컴포넌트가 전체 상태를 구독해 상태 하나가 바뀌어도 테이블 전반이 렌더링에 참여했음
  - 엔진 의존 코드를 한 경계로 모은 뒤 기능별로 이전하고, 컴포넌트마다 필요한 상태만 구독하도록 변경
  - 10,000행, 8컬럼에서 14자 검색어 입력의 누적 React 렌더 시간이 1,927ms에서 61ms로 감소(당시 동일 조건 비교, 5회 중앙값)
  - 차세대도 새 테이블 API로 이전하고, 타입 검사로 잡히지 않던 커스텀 정렬 연결을 4개 화면에서 수정

MaxGauge 기존 제품군: 고객 요청부터 패키징까지 `2026.09부터 전담`

`JavaScript / ExtJS / Playwright / Claude Code / Codex`

MFO, MFT, MFA, MFD, MFS의 레거시 업무를 전담합니다. 고객 요청의 분석부터 재현, 구현, 검증, 버전 관리와 패키징까지 맡으며 대시보드와 차세대, exem-table 개발을 병행하고 있습니다.

- **제품 지식을 AI가 참조할 수 있는 작업 기준으로 정리**
  - 처리 규칙은 Logic map에, 화면과 API의 대응은 Spec map에 정리해 AI가 기존 동작을 참고하도록 구성
  - 변경 의도를 전달하면 AI가 제품 맥락을 참고해 구현과 검증을 진행하는 방식으로 유지보수
- **반복 결함을 리뷰 스킬과 회귀 테스트에 반영**
  - 과거 MR에서 자주 발생한 버그와 사이드 이펙트를 조사해 AI 리뷰 스킬의 점검 항목으로 구성
  - 변경 커밋에 대한 리뷰 기록이 있어야 MR 작성과 패키지 전달 절차를 진행하도록 연결
  - 수정한 동작을 테스트 코드로 남겨 후속 변경에서도 회귀 여부를 확인
- **시나리오 이미지로 변경 의도를 확인하는 리뷰**
  - 변경 시나리오마다 수정 전후 화면을 MR에 첨부하도록 절차를 구성
  - AI 코드 리뷰와 테스트를 거친 변경은 ExTuner 연동 등에서 시나리오 이미지를 중심으로 확인해 병합
- **버전에 포함된 작업과 소스 기준 추적**
  - 기존에는 어떤 버전에 어떤 작업이 포함됐는지 추적하기 어려웠음
  - 작업 MR과 Client 및 SQL 커밋, 태그, 패키지 기록을 연결해 버전에 포함된 작업과 배포 소스를 추적
- **MFO에서 ExTuner 튜닝 요청과 결과 확인 연결**
  - MFO에서 SQL 튜닝 도구인 ExTuner로 요청을 보내고, 이력과 진행 상태, 보고서를 확인하는 기능 개발
  - 화면마다 SQL과 인스턴스 필드가 달라, 요청 값을 추측하는 대신 각 화면의 행 데이터를 요청 형식으로 바꾸는 규칙을 명시
  - 요청 후 응답이 없어도 접수됐을 가능성이 있으므로 실패로 확정하지 않고 '접수 확인 중' 상태로 표시
  - 로컬 MFO와 실제 연동 서버(PJS), ExTuner 모의 응답으로 요청 기록과 접수 실패, 응답 유실 시 동작을 검증

Task Analyzer: 작업 맥락 검색 도구

실험 프로젝트

`Python / TypeScript / MCP`

요구사항과 관련 코드, 과거 작업 이력을 찾는 실험에서 시작했습니다. 검색 기능을 독립 서버로 분리해 여러 AI 도구에서 사용하도록 구성했습니다.

- **검색 기능을 독립 서버로 분리**
  - Task Analyzer 외의 도구에서도 쓰도록 데이터 관계와 검색 기능을 별도 서버로 분리
  - 서버는 근거 검색을, 호출하는 AI 도구는 답변 생성을 맡도록 역할을 나눔
  - 입력과 결과 형식을 규격화해 MCP나 스킬에서도 같은 검색 API를 사용하도록 구성
- **플러그인으로 배포해 실제 작업에 활용**
  - EXEM AI Marketplace를 구성해 Claude Code와 Codex에서 같은 검색 플러그인을 설치하도록 제공
  - 회의록 정리 중 관련 태스크를 찾아 링크를 댓글에 남기는 흐름에 활용
  - 개발 작업 중 관련 이슈와 변경 이력, 문서를 찾아 작업 맥락을 확인하는 데 사용

GitLab 슬래시 커맨드: 스레드에서 AI와 협업

실험 프로젝트

`TypeScript / LangGraph`

개인 PC로 맥락을 옮기지 않고 GitLab MR 댓글에서 AI에 질문과 수정 요청을 전달하는 봇을 만들었습니다.

- **논의 중인 스레드에서 AI 호출과 결과 공유**
  - MR 댓글에서 AI를 호출하고 결과를 같은 스레드에 남겨 동료가 함께 검증하고 리뷰할 수 있도록 구현
- **스레드별 대화와 MR 공통 맥락 분리**
  - 여러 스레드의 대화가 섞이지 않도록 이력은 분리하고 MR 단위에서는 필요한 결과 요약만 공유
  - GitLab에서 서로 다른 스레드의 이력 분리와 앞선 결과 요약 전달을 확인

학력

동의대학교 정보통신공학과 / 응용 소프트웨어 공학 부전공 | 2017.03 - 2024.03

최근 글

전체 글 최신 5개 / 2026.10.24 확인

자본주의 1부 - 돈은 빚이다
2026.09.20

지금 테스트 코드가 더더욱 중요한 이유
2026.09.06

스틱 후기
2026.08.29

Tanstack Table V9 마이그레이션 회고
2026.08.26

개발자로 살아남기 - 성장에 대한 고민
2026.08.09